# Preliminary Analysis of Simulation Modules

Krzysztof Siegieda

February 1, 2026

## Purpose of this Document

This document is a technical companion to the research proposal. It records simulation modules, how to run them from the command line, and preliminary pilot observations. Results reported in Sections 2–5 are intentionally cautious because larger panels and time-series logging are still pending.

## 1 Self-Play Learning (Q-learning vs Q-learning)

### 1.1 Overview

The baseline environment is a sequential-move pricing game on a discrete grid:

$$\{0, 1/k, 2/k, \ldots, 1\}, \qquad k = 6 \text{ in the baseline.}$$

Firms alternate moves. Each firm uses tabular Q-learning where:

- the **state** is the opponent's most recent price,
- the **action** is the firm's own price choice,
- learning follows a Klein-style two-step target,
- exploration follows a decaying $\epsilon$-greedy schedule.

Late-phase outcomes are computed using the final 10% of periods.

### 1.2 How to run (baseline self-play)

Script: `self_play_runner.py` (file name may differ in your folder).

**Command template**

```
python self_play_runner.py --T 500000 --k 6 --alpha 0.3 --delta 0.95 \
  --n_seeds 1000 --seed_start 1 --csv results.csv
```

**Key parameters**

- `--T`: periods per run (learning horizon).
- `--k`: price grid resolution (k=6 gives 7 price points).
- `--alpha`, `--delta`: learning rate and discount factor.
- `--n_seeds`, `--seed_start`: number of independent runs and starting seed.
- `--csv`: output CSV filename.

## 1.3 Data visualisation (late-phase summaries)

Two figures summarise cross-sectional late-phase behaviour:

- **Figure 1**: price–profit outcome map.
- **Figure 2**: regime frequencies with 95% confidence intervals.

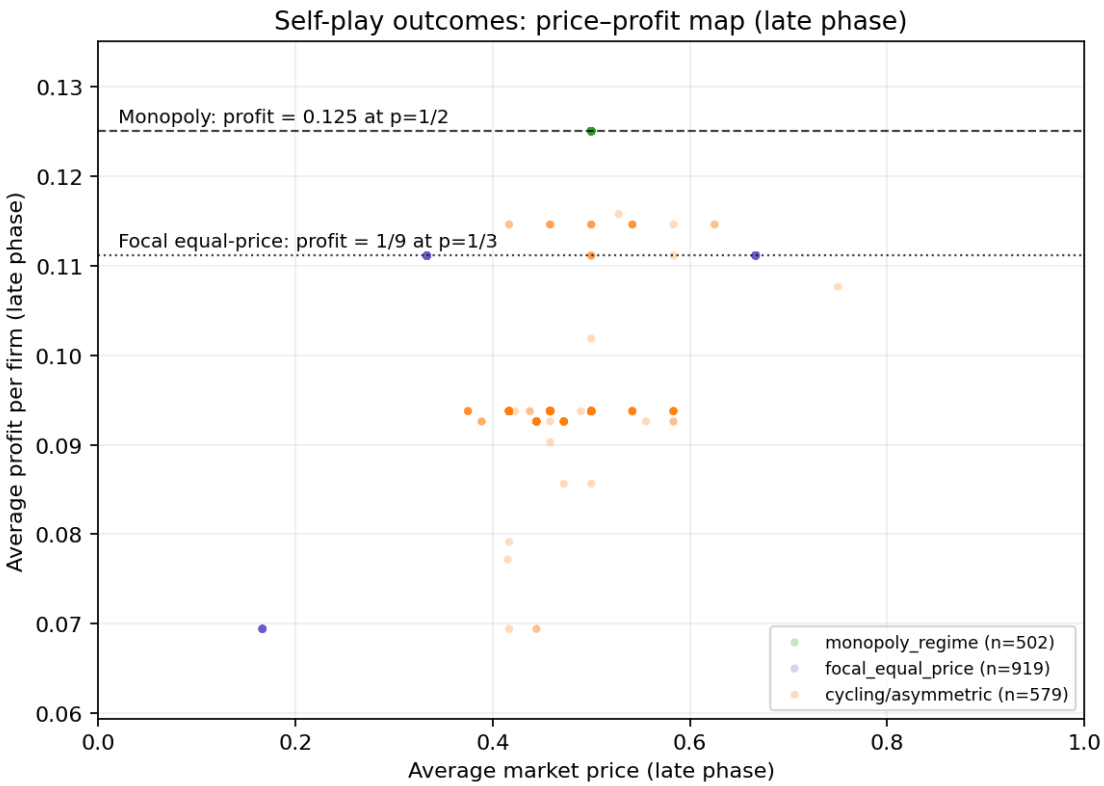


Figure 1: Price–profit outcome map (late phase).

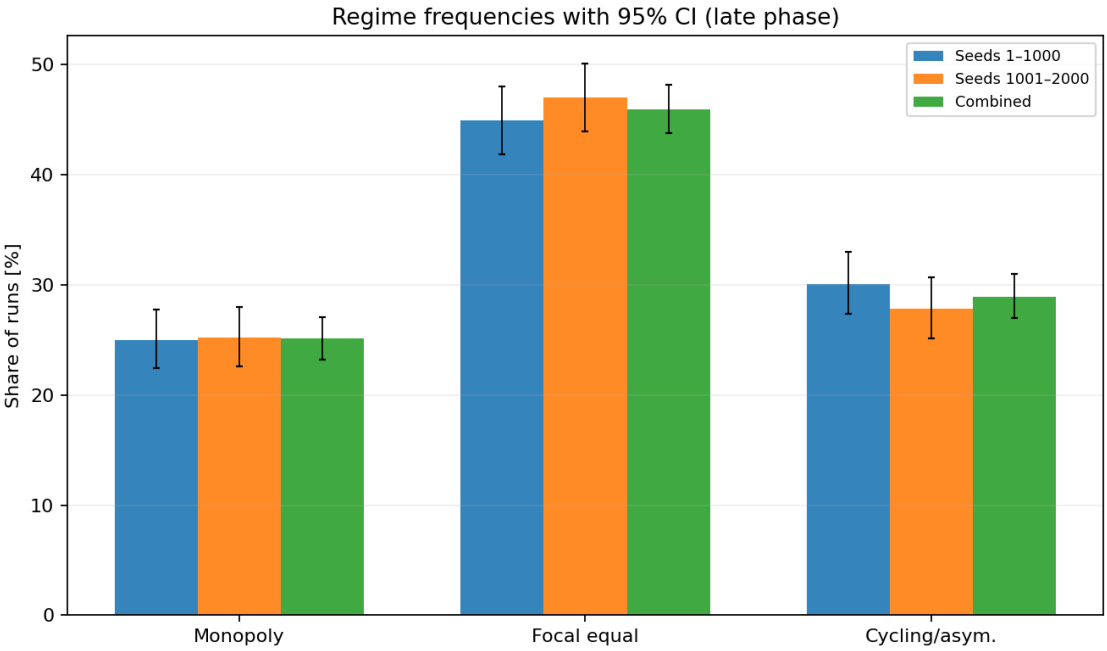


Figure 2: Regime frequencies with 95% confidence intervals.

**Remark.** These plots are based on late-phase summaries. Time-series logging will be added later to produce trajectory-based diagnostics (in the spirit of Klein-style Figures 1–3).

# 2 Memory Extensions (pilot)

This block studies asymmetric self-play where one firm has a richer informational state. The environment is unchanged; only the state definition differs.

## 2.1 Q(1-step) vs Q(2-step memory)

Script: `self_play_2step_runner.py` .

### Command template

```
python self_play_2step_runner.py --T 200000 --k 6 --alpha 0.3 --delta 0.95 \
  --n_seeds 200 --seed_start 1 --csv q1_vs_mem2.csv
```

**Key difference** Firm 2 observes the last two opponent prices (instead of only the last one).

**Pilot conclusion (non-final).** From preliminary runs it is difficult to determine a stable ranking of which learner is better. Outcomes vary across seeds and depend on the regime reached. A robust assessment requires a panel across multiple $T$ values and larger seed batches.

## 2.2 Q(1-step) vs Q(3-step memory)

Script: `self_play_3step_runner.py` .

### Command template

```
python self_play_3step_runner.py --T 100000 --k 6 --alpha 0.3 --delta 0.95 \
  --n_seeds 200 --seed_start 1 --csv q1_vs_mem3.csv
```

**Key difference** Firm 2 observes the last three opponent prices; the state space increases from $n$ to $n^3$.

**Pilot conclusion (non-final).** Because 3-step memory implies a much larger Q-table, there is a meaningful risk that default parameters are not appropriate. A credible evaluation likely requires a broader panel (potentially several horizons and multiple learning settings), including parameter tuning (e.g. smaller $\alpha$, alternative exploration schedules, longer horizons).

# 3 Q-learning vs Simple Intruders (from $t = 0$; pilot)

Firm 1 is a standard Q-learner (1-step memory). Firm 2 is replaced from the start by an intruder that maximises its own profit. Intruders:

- **MBR**: myopic best response.
- **EXP3**: stateless multi-armed bandit.
- **RBF**: rule-based follower (matches Firm 1's current price).

Script: `q_vs_intruder.py`.

### Commands

```
python q_vs_intruder.py --intruder mbr  --T 20000 --n_seeds 200 --csv out_mbr.csv
python q_vs_intruder.py --intruder exp3 --T 20000 --n_seeds 200 --csv out_exp3.csv
python q_vs_intruder.py --intruder rbf  --T 20000 --n_seeds 200 --csv out_rbf.csv
```

### Key parameters

- `--intruder`: selects the opponent: `mbr` (best response), `exp3` (bandit), `rbf` (price matcher).
- `--T`, `--n_seeds`: horizon and number of independent runs.
- (optional) `--exp3_gamma`: exploration intensity for EXP3 (only relevant with `--intruder exp3`).

**Pilot conclusion (non-final).** The intruders generate qualitatively different behaviour, but small-scale runs are not sufficient for stable claims about collusion robustness or profit dominance. Larger unified batches are required.

# 4 Two-Stage Collusion-Break Test (intruder after pre-training; pilot)

This experiment tests whether collusion-like behaviour learned in self-play survives when an intruder replaces one firm.

## 4.1 Design

- **Stage A**: Q-learning vs Q-learning for $T_{\text{train}}$ (pretraining).
- **Stage B**: replace Firm 2 with an intruder for $T_{\text{test}}$ and observe whether high-price behaviour persists.

Script: `collusion_break_test.py`.

### Command template

```
python collusion_break_test.py --intruder mbr \
  --T_train 50000 --T_test 50000 \
  --k 6 --alpha 0.3 --delta 0.95 \
  --n_seeds 200 --seed_start 1 \
  --csv break_mbr.csv
```

### Key parameters (what matters)

- `--intruder`: `mbr`, `exp3`, or `rbf` (intruder in Stage B).
- `--T_train`, `--T_test`: lengths of Stage A and Stage B.
- `--alpha`, `--delta`, `--k`: learning/environment parameters.
- `--n_seeds`, `--seed_start`: number of runs and seed range.

### Firm 1 behaviour in Stage B

- `--q_continue_learning`: if set, Firm 1 keeps updating Q during Stage B; otherwise Firm 1 plays its learned policy (frozen Q-table).
- `--q_eps_test`: exploration rate for Firm 1 in Stage B (default 0 = greedy).

### Collusion-break thresholds (pilot indicator)

- `--break_share_threshold`: required share of $(0.5, 0.5)$ to call Stage A monopoly-like.
- `--break_price_threshold`: required average price to call Stage A monopoly-like.

### Optional: Stage-B time series output

- `--save_ts`: save a downsampled time series for Stage B.
- `--ts_stride`: keep every N-th step (smaller file).
- `--ts_prefix`: prefix for the saved time-series CSV.

**Pilot conclusion (non-final).** Earlier tests indicate that collusion can be broken, but outcomes depend on the specific run: some seeds collapse to low prices while others maintain high prices. The devil is in the details. A final assessment requires larger panels and trajectory-based diagnostics.

**Next steps (core point).** The most important follow-up is time-series diagnosis in Stage B: the analysis will move beyond late-phase averages and examine *how* breaks occur (immediately, gradually, or not at all). Those trajectory-based results will provide the primary figures for the final paper.